

Normalization Report - Hotel Reservation System

Project: P4_Hotel_Reservation_System

Database: P4_HotelReservation

Date: 2026

1. Introduction

This document explains the normalization process applied to the Hotel Reservation System database. Normalization eliminates data redundancy, ensures data integrity, and prevents anomalies. The database is normalized up to Boyce-Codd Normal Form (BCNF).

2. Unnormalized Form (UNF)

Before normalization, all data was stored in a single flat table.

Raw Table Example:

BookingID	Customer	Phone	RoomNo	Price	CheckIn	CheckOut	Total
1	Rishabh	9876543210	101	5000	2026-06-01	2026-06-03	10000
1	Rishu	9876543210	102	5000	2026-06-01	2026-06-03	10000
2	Rolly	9876543211	201	3000	2026-06-05	2026-06-07	6000

Problems Identified:

- Repeating customer data for multiple bookings
- Phone number stored multiple times
- Update anomaly: Changing phone number requires updating multiple rows
- Insert anomaly: Cannot add customer without a booking
- Delete anomaly: Deleting booking removes customer information

3. First Normal Form (1NF)

Rule Applied: Eliminate repeating groups. Each cell must contain a single value. Each record must be unique.

1NF Table Structure:

BookingID	Customer	Phone	RoomNo	Price	CheckIn	CheckOut	Total
1	Raj	9876543210	101	5000	2026-06-01	2026-06-03	10000
2	Priya	9876543211	201	3000	2026-06-05	2026-06-07	6000
3	Amit	9876543212	202	3000	2026-06-10	2026-06-12	6000

+-----+-----+-----+-----+-----+-----+-----+

Still Remaining Problems:

- Partial dependency: Customer details depend on BookingID, not on RoomNo
- Transitive dependency: Price depends on RoomNo, not on BookingID
- Data redundancy still exists

4. Second Normal Form (2NF)

Rule Applied: Remove partial dependencies. All non-key attributes must depend on the entire primary key.

Solution: Split into multiple tables.

Table A: Customers

+-----+-----+-----+		
id	name	phone
+-----+-----+-----+		
1	Raj	9876543210
2	Priya	9876543211
3	Amit	9876543212
4	Neha	9876543213
+-----+-----+-----+		

Primary Key: id

Table B: Rooms

+-----+-----+-----+			
id	room_no	price	status
+-----+-----+-----+			
101	101	5000	Available
102	102	5000	Available
103	201	3000	Available
104	202	3000	Available
105	301	8000	Available
+-----+-----+-----+			

Primary Key: id

Table C: Bookings

+-----+-----+-----+-----+-----+					
id	cust_id	room_id	checkin	checkout	total
+-----+-----+-----+-----+-----+					
1	1	101	2026-06-01	2026-06-03	10000
2	2	103	2026-06-05	2026-06-07	6000
3	3	104	2026-06-10	2026-06-12	6000
+-----+-----+-----+-----+-----+					

Primary Key: id

Foreign Keys: cust_id references Customers(id), room_id references Rooms(id)

Now in 2NF: All non-key attributes depend on entire primary key.

5. Third Normal Form (3NF)

Rule Applied: Remove transitive dependencies. No non-key attribute should depend on another non-key attribute.

Analysis for 3NF:

Customers Table:

- name depends only on id
- phone depends only on id
- No transitive dependency

Rooms Table:

- room_no depends only on id
- price depends only on id
- status depends only on id
- No transitive dependency

Bookings Table:

- cust_id depends only on id
- room_id depends only on id
- checkin depends only on id
- checkout depends only on id
- total depends only on id
- No transitive dependency

Result: Database is in 3NF.

6. Boyce-Codd Normal Form (BCNF)

Rule Applied: For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.

Checking each table for BCNF:

Customers Table:

- Functional dependencies: $id \rightarrow name, phone$
- id is the primary key (superkey) ✓
- phone is unique but does not determine other non-key attributes ✓
- Result: In BCNF ✓

Rooms Table:

- Functional dependencies: $id \rightarrow room_no, price, status$
- $room_no \rightarrow id$ (room_no is unique, making it a candidate key)

- All determinants are superkeys ✓
- Result: In BCNF ✓

Bookings Table:

- Functional dependencies: $id \rightarrow cust_id, room_id, checkin, checkout, total$
- $(cust_id, room_id, checkin)$ is a composite candidate key
- No non-key attribute determines another non-key attribute ✓
- Result: In BCNF ✓

BCNF Verification Table:

Table	Candidate Keys	BCNF Status
Customers	id	Satisfied ✓
Rooms	id, room_no	Satisfied ✓
Bookings	id, (cust_id, room_id, checkin)	Satisfied ✓

Conclusion: All tables are in BCNF. No further decomposition needed.

7. Final Database Schema

Table 1: Customers

Column	Data Type	Constraints
id	INT	PRIMARY KEY, IDENTITY
name	VARCHAR(30)	NOT NULL
phone	VARCHAR(10)	NOT NULL, UNIQUE

Table 2: Rooms

Column	Data Type	Constraints
id	INT	PRIMARY KEY, IDENTITY
room_no	INT	NOT NULL, UNIQUE
price	INT	NOT NULL
status	VARCHAR(10)	DEFAULT 'Available'

Table 3: Bookings

Column	Data Type	Constraints
id	INT	PRIMARY KEY, IDENTITY

cust_id	INT	FOREIGN KEY	
room_id	INT	FOREIGN KEY	
checkin	DATE	NOT NULL	
checkout	DATE	NOT NULL	
total	INT	-	
+-----+-----+-----+			

8. Relationship Diagram

Customers (1) -----< (M) Bookings

Rooms (1) -----< (M) Bookings

Explanation:

- One customer can make many bookings (One-to-Many)
- One room can have many bookings over time (One-to-Many)
- Each booking belongs to exactly one customer and one room

9. Benefits Achieved

+-----+-----+	
Problem	Solution
+-----+-----+	
Data redundancy	Eliminated by separate tables
Update anomalies	Update customer information only once
Insert anomalies	Add a customer without making a booking
Delete anomalies	Delete a booking without losing customer data
Referential integrity	Enforced by foreign key constraints
+-----+-----+	

10. Normalization Summary

+-----+-----+		
Normal Form	Status	Applied?
+-----+-----+		
1NF	Achieved	No repeating groups
2NF	Achieved	No partial dependencies
3NF	Achieved	No transitive dependencies
BCNF	Achieved	Every determinant is a superkey
+-----+-----+		

11. Conclusion

The Hotel Reservation System database successfully achieves:

- ✓ First Normal Form (1NF) - No repeating groups

- ✓ Second Normal Form (2NF) - No partial dependencies
- ✓ Third Normal Form (3NF) - No transitive dependencies
- ✓ Boyce-Codd Normal Form (BCNF) - All determinants are superkeys
- ✓ All integrity constraints applied
- ✓ Ready for production use